

Project outline:

1. Closed Mesh Input:
 - a. Normalize scale, verify watertightness/orientation, sample the surface, store normals, and keep sample-to-triangle provenance.
2. 3D Delaunay Tetrahedralization
 - a. Generalize from 2D triangles to 3D tetrahedra. Your history DAG idea still applies. The hard parts are robust orient3d and inSphere.
3. Voronoi Dual:
 - a. Each tetrahedron gives a Voronoi vertex/circumcenter. Adjacent tetrahedra define Voronoi faces/edges. Store candidate sphere center, radius, nearest/contact surface samples.
4. Interior/Exterior Classification:
 - a. Use winding number, signed distance, or robust ray casting. Keep interior candidates for the inner medial axis.
5. Pole Selection / Medial Candidate Selection":
 - a. This is the important correction: don't treat all interior Voronoi vertices as medial-axis points. For each surface sample, select meaningful poles, usually far Voronoi vertices in inward/outward normal directions. This is where Power Crust-style logic enters.
6. Build the Medial Surface Complex
 - a. Produce a structure with vertices, edges, triangles/sheets, radii, contact sets, and confidence/stability values. The goal is not just points; it is a sheet complex.
7. Filtering / Simplification:
 - a. Apply radius thresholds, contact-angle tests, scale-axis filtering, stability under resampling, or persistence-style pruning. You'll likely iterate here a lot.
8. Visualization:
 - a. Separate layer. Show mesh, samples, normals, Voronoi vertices, poles, medial balls, sheet triangles, singular edges, spokes to contact points, and filtering stages.
9. CGAL / Other Libraries as Oracle:
 - a. Use CGAL or another tool to compare behavior, not as the main implementation.

Useful Links

Core Path

1. Robust Predicates

Shewchuk — Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates

Essential for `orient3d`, `inSphere`, and avoiding broken Delaunay behavior.

- <https://www.cs.cmu.edu/~quake/robust.html>
-

2. 3D Delaunay Reference Implementation

CGAL — Delaunay_triangulation_3 Documentation

Useful as an oracle for your tetrahedralization.

- https://doc.cgal.org/latest/Triangulation_3/
-

3. Voronoi Poles / Surface Reconstruction

Amenta, Bern, Kamvysselis — A New Voronoi-Based Surface Reconstruction Algorithm

Important for understanding poles and local feature size.

- <https://people.eecs.berkeley.edu/~jrs/meshpapers/AmentaBernKamvysselis.pdf>
-

4. Power Crust / Polar Balls / MAT

Amenta, Choi, Kolluri — The Power Crust, Unions of Balls, and the Medial Axis Transform

Probably the most directly relevant paper for this project.

- <https://www.cs.ucdavis.edu/~amenta/pubs/power.pdf>
-

5. Voronoi Subcomplex Approximation

Dey, Zhao — Approximate Medial Axis as a Voronoi Subcomplex

Important because raw 3D Voronoi vertices are not sufficient.

- <https://www.sciencedirect.com/science/article/pii/S0010448503000617>

6. 3D Medial-Axis Structure

Giblin, Kimia — A Formal Classification of 3D Medial Axis Points and Their Local Geometry

Useful for understanding:

- Sheets
 - Seams
 - Junction curves
 - Special contact types
 - <https://pubmed.ncbi.nlm.nih.gov/15376898/>
-

Filtering / Stability

7. Scale Axis Transform

Miklos, Giesen, Pauly — Discrete Scale Axis Representations for 3D Geometry

Useful for pruning unstable medial artifacts while preserving shape.

- <https://www.cs.princeton.edu/courses/archive/spring11/cos598A/pdfs/Miklos10.pdf>
-

8. Unstable Flow Complex

Dey et al. — Medial Axis Approximation and Unstable Flow Complex

More theoretical, but useful for topology guarantees.

- https://www.cs.toronto.edu/~sadri/publications/medial_new.pdf
-

9. Q-MAT

Q-MAT — Computing Medial Axis Transform by Quadratic Error Minimization

Good modern reference for simplifying medial meshes.

- <https://binwangthss.github.io/qmat/qmat.html>
-

10. Q-MAT+

Error-controllable and feature-sensitive simplification.

- <https://www.sciencedirect.com/science/article/abs/pii/S0167839619300214>
-

Modern Medial Mesh / Power Diagram Work

11. Medial Meshes

Medial Meshes: A Compact and Accurate Representation of Medial Axis Transform

Useful for thinking about MAT as:

- Spheres
 - Cones
 - Slabs
 - <https://i.cs.hku.hk/~yzyu/publication/medialmesh-tvcg2016.pdf>
-

12. Feature-Preserving MAT via Restricted Power Diagram

Relevant for preserving:

- Sharp edges
 - Seams
 - Junctions
 - <https://arxiv.org/abs/2210.13676>
-

13. MATTopo

Topology-preserving MAT using restricted power diagrams.

- <https://arxiv.org/abs/2403.18761>
-

14. LSMAT

Least-squares MAT with robust fitting/optimization ideas.

- <https://arxiv.org/abs/2010.05066>
-

Mesh Input / Classification

15. Fast Winding Numbers

Useful for robust inside/outside classification on imperfect meshes or triangle soups.

- <https://www.dgp.toronto.edu/projects/fast-winding-numbers/>
-

16. Generalized Winding Numbers

Original robust inside/outside segmentation work.

- <https://repository.upenn.edu/hms/157>
-

17. Geometry Central Surface Sampling

- https://geometry-central.net/surface/algorithms/surface_sampling/
-

Libraries / Oracles

18. CGAL 3D Alpha Shapes

Useful for Delaunay-based filtering and shape reconstruction ideas.

- https://doc.cgal.org/latest/Alpha_shapes_3/
-

19. CGAL Surface Mesh Skeletonization

Not a full MAT implementation, but useful for curve-skeleton comparison.

- https://doc.cgal.org/latest/Surface_mesh_skeletonization/
-

20. TetGen Manual

Useful practical tetrahedralization reference.

- <https://www.wias-berlin.de/software/tetgen/1.5/doc/manual/manual.pdf>
-

Suggested Reading Order

1. Shewchuk
2. CGAL Delaunay docs
3. Amenta / Bern / Kamvysselis
4. Power Crust
5. Dey / Zhao
6. Scale Axis Transform
7. Q-MAT / Medial Meshes

After the first prototype, move into restricted power diagram papers.

Use Existing Code Only For

- Mesh loading
- Vector/matrix math
- Visualization
- Optional oracle comparison
- Possibly robust predicates

Avoid numerical robustness dominating the project unless that is your explicit focus.

Do **not** begin by cloning Power Crust, Q-MAT, MATTopo, etc.

Those systems are too complete and you will spend more time decoding engineering decisions than learning the geometry.

Suggested Starter Project Layout

```
medial-axis-lab/  
src/  
  core/  
    vec3.h  
    mesh.h  
    surface_sample.h  
    delaunay3.h  
    voronoi_dual.h  
    medial_candidate.h  
    medial_complex.h  
    filters.h  
  
  viewer/  
    debug_viewer.h  
    polyscope_viewer.cpp  
  
  app/  
    inspect.cpp  
  
  tests/  
    sphere_tests.cpp  
    cube_tests.cpp  
    cylinder_tests.cpp
```

Initial Pipeline Skeleton

```
Mesh mesh = loadMesh(path);  
mesh.normalizeUnitBox();  
mesh.assertClosedAndOriented();  
  
std::vector<SurfaceSample> samples =  
    sampleSurface(mesh, options);  
  
Delaunay3 delaunay;  
delaunay.insert(samples);
```

```
VoronoiDual voronoi =  
    buildVoronoiDual(delaunay);  
  
std::vector<MedialCandidate> candidates =  
    extractInteriorVoronoiCandidates(voronoi, mesh);  
  
std::vector<MedialCandidate> poles =  
    selectPoles(samples, candidates);  
  
MedialComplex medial =  
    buildMedialComplex(poles, voronoi);  
  
filterUnstableBranches(medial, filterOptions);  
  
viewer.show(mesh, samples, voronoi, poles, medial);
```

Recommended Dependencies

- Polyscope for visualization
<https://polyscope.run/>
 - tinyobjloader or Assimp for mesh loading
 - Shewchuk predicates for `orient3d` / `inSphere`
<https://www.cs.cmu.edu/~quake/robust.html>
 - CGAL only as an oracle
https://doc.cgal.org/latest/Triangulation_3/
-

Recommended Learning Progression

1. Start with Analytic Shapes

Test on:

- Sphere
- Capsule
- Cylinder
- Box

- Torus-like shapes

Before arbitrary meshes, understand what the medial axis should approximately look like.

2. Build Debug Views Early

Visualize:

- Samples
- Normals
- Delaunay tetrahedra
- Voronoi centers
- Interior centers
- Poles
- Balls
- Spokes
- Sheet triangles

This is where learning happens fastest.

3. Implement a Naive Version First

Even if slow.

A slow, mostly-correct implementation is better than a fast confusing one.

4. Use Papers as Behavioral Targets

Read:

- Power Crust
- Dey-Zhao
- Scale Axis

...but implement your own simplified version first.

5. Only Borrow Primitives

It is completely reasonable to reuse:

- Robust predicates
- Mesh I/O
- Visualization support

Those are not the core intellectual part of the project.